

Reviewer Pack — Matroid Rank Gap in Monotone DNF for k-CLIQUE

Entry 202b457c781c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 04:29:55 UTC

Matroid Rank Gap in Monotone DNF for k-CLIQUE

Entry ID: 202b457c781c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 04:29:55 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone DNF Formulas for k-CLIQUE

Statement:

For a k-CLIQUE indicator function expressed as a monotone DNF with m clauses, the rank of the matroid formed by its clause vectors is $\Omega(n)$ if $m \leq n^{O(1)}$, and $O(\log n)$ if $m \geq n^{\Omega(k \{1/4\})}$.

Rationale (proposer’s reasoning):

Matroid rank captures combinatorial dependencies in clause interactions, which may expose structural barriers to succinct DNF representations. The conjecture ties this invariant to known lower bounds for k-CLIQUE, suggesting rank gaps could amplify hardness.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9de6352250f75a9f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i + max(range(i, rows), key=lambda x: abs(matrix[x][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            for j in range(cols):
                if j != i:
                    factor = matrix[j][i] / matrix[i][i]
                    for k in range(rows):
                        matrix[j][k] -= factor * matrix[i][k]
        rank = sum(1 for row in matrix if any(row))
        return rank

    def dnf_from_3cnf(cnf):
        dnf = []
        for clause in cnf:
            dnf.append([abs(lit) for lit in clause])
        return dnf

    def generate_random_3cnf(n, m):
        variables = list(range(1, n + 1))
        cnf = []
        for _ in range(m):
            clause = random.sample(variables, 3)
            cnf.append(clause)
        return cnf

    n = 40
    k = 10
    m_values = [5, 10, 15, 20, 30, 40]

    results = []
    for m in m_values:
        cnf = generate_random_3cnf(n, m)
        dnf = dnf_from_3cnf(cnf)
        matrix = [[int(abs(lit) == var) for var in range(1, n + 1)] for clause in dnf]
        rank = gaussian_elimination(matrix)

        results.append({
```

```

        "n": n,
        "m": m,
        "rank": rank
    })

total_rank = sum(result["rank"] for result in results)
mean_rank = total_rank / len(results)
conjecture_holds = all(mean_rank >= 0.8 * n if m <= n ** (1/2) else mean_rank <= 5 * math.log

return {
    "metric_name": "Matroid Rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 50))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='rank_gap' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115ac035.py", line 83, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115ac035.py", line 56, in run_trial

```

    matrix = [[int(abs(lit) == var) for var in range(1, n + 1)] for clause in dnf]
    ~~~

```

NameError: name 'lit' is not defined. Did you mean: 'List'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to NameError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the NameError in test_115ac035.py by defining 'lit' variable before usage

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	93373
2	preregistration	ollama_remote	qwen3:8b	0	0	31639
3	novelty	ollama_remote	qwen3:8b	0	0	27418
4	novelty	ollama_remote	qwen3:8b	0	0	21250
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16033
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12584
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11004
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9296
9	judge	ollama_remote	qwen3:8b	0	0	21869

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 244466 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/202b457c781c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/202b457c781c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/202b457c781c.tar.gz` (if generated)